

WEEK-10 ELASTIC BEAN

Step 1: Open AWS Elastic Beanstalk

1. Login to the **Amazon Web Services (AWS) Console**
 2. Go to **Compute** section
 3. Search for **Elastic Beanstalk**
 4. Click **Create Application**
-

Step 2: Create Application

1. Enter **Application Name** (e.g., *MyApp*)
 2. Add **Description** (optional)
 3. Click **Create**
-

Step 3 Create Environment

1. Click **Create Environment**
 2. Choose:
 - **Web Server Environment** (for web applications)
 3. Click **Select**
-

Step 4: Configure Environment

1. Enter **Environment Name**
 2. Choose **Platform**:
 - Tomcat (for .war files)
 - OR Node.js / Python / PHP / etc.
 3. Under **Application Code**:
 - Select **Upload your code**
 - Upload your .war or .zip file
-

Step 5: Configure Service Access

1. **Service Role** → Select existing (e.g., *LabRole*)
 2. **EC2 Key Pair** → Optional (used for SSH access)
 3. **EC2 Instance Profile** → Select (e.g., *LabInstanceProfile*)
-

Step 6: Configure Network

1. Select **VPC** (default is fine)
2. Select **Subnets**
3. Enable **Public IP**

Step 7: Instance & Scaling Setup

1. Choose **Instance Type** (e.g., *t2.micro* – free tier)
 2. Change environment type to **Load Balanced**
 3. Set:
 - **Minimum instances** → 1
 - **Maximum instances** → 2 (or more)
 4. Enable **Load Balancer** (recommended)
 5. Leave remaining settings as default
 6. Click **Skip to Review**
-

Step 8: Review & Create

1. Click **Review**
 2. Click **Create**
-

Deployment Process (Automatic by AWS)

AWS automatically creates:

- EC2 Instances
 - Load Balancer
 - Auto Scaling Group
 - Security Groups
-

Access Your Application

1. After deployment, you will get a **URL (domain link)**
 2. Open it in a browser
 3. Your application will run
-

Week 7(lamda dymno db)

1 Create an S3 Bucket (Event Source)

1. Log in to the **Amazon Web Services Console**.
2. Go to **Amazon S3** service.
3. Click **Create bucket**.
4. Enter:
 - **Bucket name:** s3-lambda-trigger-bucket (any unique name).
 - **Region:** Choose your preferred region.
5. Keep other settings default.
6. Click **Create bucket**.

This bucket will trigger the Lambda function when a file is uploaded.

2 Create a Dynamo DB Table

1. Open **Amazon Dynamo DB**.
2. Click **Create table**.
3. Enter details:
 - **Table name:** newtable
 - **Partition key:** unique
 - **Data type:** String
4. Keep other settings default.
5. Click **Create table**.

This table will store S3 event details.

3 Create Lambda Function

1. Open **AWS Lambda**.
2. Click **Create function**.
3. Select **Author from scratch**.

Fill the details:

- **Function name:** S3DynamoLambda
- **Runtime:** Python 3.x
- **Execution role:** Use existing role → LambdaS3DynamoRole

Click **Create function**.

5 Add the Python Code

In **Lambda** → **Code section**, paste your code:

```

import boto3
from uuid import uuid4

def lambda_handler(event, context):
    s3 = boto3.client("s3")
    dynamodb = boto3.resource('dynamodb')

    for record in event['Records']:
        bucket_name = record['s3']['bucket']['name']
        object_key = record['s3']['object']['key']
        size = record['s3']['object'].get('size', -1)
        event_name = record['eventName']
        event_time = record['eventTime']

        dynamoTable = dynamodb.Table('newtable')

        dynamoTable.put_item(
            Item={
                'unique': str(uuid4()),
                'Bucket': bucket_name,
                'Object': object_key,
                'Size': size,
                'Event': event_name,
                'EventTime': event_time
            }
        )

```

Click **Deploy**.

6 Add S3 Trigger to Lambda

1. In Lambda function page.
2. Click **Add Trigger**.
3. Select **S3**.
4. Choose your bucket:
s3-lambda-trigger-bucket
5. Event type:
 - **All object create events**
6. Check **Enable trigger**.
7. Click **Add**.

7 Test the System

1. Open **S3 bucket**.
2. Upload any file (example: test.txt).
3. The upload event will trigger the Lambda function.

8 Verify DynamoDB Entry

1. Open **DynamoDB** → **Tables** → **newtable**.

2. Click **Explore table items**.
3. You will see a record like:

unique	Bucket	Object	Size	Event	EventTime
UUID	bucket name	file name	size	ObjectCreated	time

✓ This confirms that **S3 event triggered Lambda and updated DynamoDB**.

STEP-9 : Monitoring using cloud watch

- Click on the Monitor tab in the lambda function
- Click on the view cloudwatch logs
- Click on the time lines and check the logs

WEEK-11—Amazon LEX

Step 1: Open Amazon Lex

1. Login to AWS Console
2. Search **Amazon Lex**
3. Click **Create bot**

Step 2: Create Bot

1. Choose:
 - **Create a blank bot**
2. Enter:
 - Bot name: HotelBookingBot
3. IAM role → Create new role
4. Remaining default → next
5. Language: English
6. Voice (optional)
7. Click **Done**

Step 3: Create Intent

1. Go to **Intents**
2. Name: BookHotel
3. Click **Create**

Step 4: Add Sample Utterances

Add user sentences (how user talks):

I want to book a hotel

Book a room

Reserve hotel

I need a room

These are **Utterances**

Step 5: Create Slots (User Inputs)

Slots collect user information.

Add Slot

1. Click **Add slot**
2. Fill:
 - Slot name: age
 - Slot type: AMAZON.Number
 - Prompt: *Which city do you want?*

Mark as **Required**

Add IF Conditions (Conditional Logic)

Used to control flow based on slot values

Example Condition:

If {age} < 18

Steps:

1. Go to **Intent** → **Slots** -> **under Advance options**
2. Scroll to **Slot capture: success response** -> **Conditional branching**
3. Click **Add condition**

Example:

Condition:

If {age} < 18

Response:

Your not eligible for hoot booking?

Step 6: Create Slots (User Inputs)

Slots collect user information.

Add Slot

3. Click **Add slot**
4. Fill:
 - Slot name: location
 - Slot type: AMAZON.City
 - Prompt: *Which city do you want?*
5. Mark as **Required**

Repeat for other slots.

1. Click **Add slot**
 2. Fill details:
 - **Slot name:** checkin
 - **Slot type:** AMAZON.Date
 - **Prompt:**
"What is your check-in date?"
 3. Mark as **Required**
(Optional but recommended)
 - Add **Retry prompt:**
"Please provide a valid date (e.g., 2026-03-25)"
 4. Click **Save**
-

Slot 7: Number of Nights (nights)

Steps:

1. Click **Add slot** again
 2. Fill details:
 - **Slot name:** nights
 - **Slot type:** AMAZON.Number
 - **Prompt:**
"How many nights will you stay?"
 3. Mark as **Required**
 4. Click **Save**
-

Step 7: Create Custom Slot Type

Save intent

Click on Back to intents

If you want custom values:

1. Go to **Slot Types**
2. Click **Add slot type** ->add blank slot type
3. Name: RoomType

Add values:

Single

Double

Suite

Save slot type

Now use this in slot: go back to intent

- Slot name: RoomType
- Slot type: RoomType

Step 9: Add Response Cards (Card Groups)

Used for buttons (quick replies)

Steps:

1. Go to **slot prompts** -> **more prompt options**
2. Click **Add** -> **add card groups**

Example:

Card Title: Select Room Type

click on add Buttons:

- Single → "Single"
- Double → "Double"
- Suite → "Suite"

Step 10: Configure Responses

Initial Response:

Welcome to Hotel Booking! What is your name?

Confirmation:

Do you want to confirm booking in {location} for {nights} nights?

Step 11: Build & Test

1. Click **Build**
2. Use **Test chatbot** panel
3. Try:

Book a hotel

Sample output

User: Book a hotel

Bot: What is your age?

User: 25

Bot: Which city do you want?

User: Hyderabad

Bot: Check-in date?

User: Tomorrow

Bot: Number of nights?

User: 2

Bot: Select room type

User: Double

Bot: Do you want to confirm booking in Hyderabad for 2 nights?

User: Yes

Bot: Booking confirmed.

Week-12-IAM

Part A: Administrative Setup (IAM User Creation)

Step 1: Login as Root User

Open AWS Management Console and sign in using root credentials.

Step 2: Open IAM Service

Search IAM and navigate to Security, Identity & Compliance → IAM.

Step 3: Create IAM User

Go to Users → Create user.

Enter username: S3_Specialist.

Enable console access and set custom password.

Step 4: Set Permissions

Select Attach policies directly.

Choose AmazonS3FullAccess.

Step 5: Review & Create

Create user and download CSV file.

Step 6: Verify User Creation

Confirm user is created.

Step 7: Note Account ID

Copy 12-digit AWS Account ID.

Step 8: Logout Root User

Step 9: Login as IAM User

Use sign-in URL and login with IAM credentials.

Testing IAM Permissions

Test 1: EC2 Access

Try accessing EC2 → Access Denied (Expected).

Test 2: S3 Access

Create S3 bucket → Success.

Part B: CLI Access

Step 10: Generate Access Keys

Go to IAM → User → Security Credentials → Create Access Key.

Step 11: Install AWS CLI

Step 12: Configure CLI

Run: aws configure

Enter keys, region ap-south-1, output json.

CLI Verification

Step 13: Test EC2 Access

Command: aws ec2 describe-instances

Result: Access Denied.

Step 14: Test S3 Access

Command: aws s3 ls

Result: Lists buckets.

Step 15: Create Bucket

Command: aws s3 mb s3://your-unique-bucket-name

Step 16: Verify in Console

Check S3 console for bucket.

Conclusion:

IAM policies apply to both GUI and CLI enforcing least privilege.

Week 13: Attaching a Role to an EC2 Instance

Step 1: Create IAM Role

1. Go to **IAM** in the AWS Console
 2. Click **Roles**
 3. Click **Create Role**
-

Step 2: Select Trusted Entity

1. Choose **AWS Service**
 2. Under **Service or use case**, select **Amazon EC2**
 3. This means:
 - You are allowing EC2 to use this role
 4. Click **Next**
-

Step 3: Add Permissions

1. Search for **AmazonS3FullAccess**
2. Select it

☞ This gives full access to **Amazon S3**

3. Click **Next**
-

Step 4: Name and Create Role

1. Enter Role Name (e.g., EC2-S3-ReadOnly-Role)
2. Review settings
3. Click **Create Role**

✓ Role is now created successfully

Step 5: Create EC2 Instance

1. Go to **Amazon EC2 Dashboard**
 2. Click **Launch Instance**
 3. Configure instance (AMI, instance type, key pair, etc.)
 4. Launch the instance
-

Step 6: Connect to EC2 Instance

1. Select the running instance
 2. Click **Connect** (SSH)
 3. Login to the instance
-

Step 7: Attach IAM Role to EC2 Instance

1. Go to **EC2 Dashboard**
 2. Select your running instance
 3. Click:
 - **Actions** → **Security** → **Modify IAM Role**
 4. Select the role you created (EC2-S3-ReadOnly-Role)
 5. Click **Update IAM Role**
-

Step 8: Verify Role Using CLI

1. Connect to your EC2 instance (SSH)
2. Run command:

```
aws s3 ls
```

✓ Result:

- Command works successfully
 - Shows S3 buckets
-

Step 9: Test Permissions Restriction

Run:

```
aws ec2 describe-instances
```

✗ Result:

- **Unauthorized error**

🔍 Reason:

- Role only has **S3 access**, not EC2 permissions

WEEK-9- Implementation of Elastic Load Balancer with Auto Scaling using AWS EC2 Instances.

Elastic Load Balancer (ALB)

Step 1: Launch Two EC2 Instances

1.1 Launch EC2 Instance #1

1. Go to **AWS EC2 Console** → **Launch Instance**.
2. **Name:** webserver-1
3. Choose an **AMI** (Amazon Linux 2).
4. Select **Instance Type** (e.g., t2.micro).
5. Add **Storage** (default is fine).
6. Configure **Security Group**:
 - Allow **SSH (22)** from your IP
 - Allow **HTTP (80)** from anywhere (0.0.0.0/0)
7. **Launch** → Select/Create Key Pair → Launch.

Run these commands after connecting VM

```
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd

echo "This is Server 1" > /var/www/html/index.html
```

✓ Now you create second **EC2 instances** running Apache, with message **This is Servier 2** each serving a simple web page.

Repeat it for second instance

```
yum update -y
yum install httpd -y
systemctl start httpd
systemctl enable httpd

echo "This is Server 2" > /var/www/html/index.html
```

Step 2: Create a Load Balancer

AWS provides **Elastic Load Balancer (ELB)**. We will create an **Application Load Balancer (ALB)**.

2.1 Configure Security Group for Load Balancer

- Go to **EC2 → Security Groups → Create Security Group**
 - Name: lb-sg
 - Allow **HTTP (80)** from anywhere
 - Optional: HTTPS (443) if using SSL

2.2 Create the Load Balancer

1. Go to **EC2 → Load Balancers → Create Load Balancer → Application Load Balancer**
2. **Basic Configuration:**
 - Name: my-alb
 - Scheme: Internet-facing
 - IP address type: IPv4
3. **Listeners:**
 - Default: HTTP (80)

2.3 Configure Availability Zones

- Select the default **VPC** where your EC2 instances exist
- Select **two /all subnets** (one for each instance if possible)

2.4 Configure Security Groups

- Choose the **lb-sg** security group created earlier

2.5 Configure Target Groups

1. **Create a new target group:**
 - Name: web-servers-tg
 - Target type: Instance
 - Protocol: HTTP
 - Port: 80
 - Health checks: / (default)
-

2. Register Targets:

- Select EC2 INSTANCES HERE --- **webserver-1** and **webserver-2**
 - Click **Include as pending** → **Register**
-

2.6 Review and Create

- Review all settings → Click **Create Load Balancer**
 - AWS will take a few minutes to provision the ALB
-

Step 3: Verify Load Balancer

1. Go to **Load Balancers** → **Select ALB** → **Description** → **DNS name**
 2. Copy the **DNS name** (something like my-alb-123456.us-east-1.elb.amazonaws.com)
 3. Paste it in a browser → You should see either **WebServer-1** or **WebServer-2** page.
 - Refresh multiple times → Load balancer distributes traffic between the two instances
-

Step 4: Optional – Test Health Checks

- Go to **Target Groups** → **web-servers-tg** → **Targets**
- Ensure **Status** of both instances is healthy

If a server is unhealthy, check the bootstrapping script or security group rules.

Step-by-Step: Auto Scaling Group with Load Balancer

Step 1: Create an AMI from Existing EC2 Instance

1. Go to **EC2 Console** → **Instances**
2. Select the EC2 instance you already configured with bootstrapping
3. Click **Actions** → **Image and Templates** → **Create Image**
4. **Provide Image Name:** e.g., my-webserver-AMI
5. Optional: Add description
6. Click **Create Image**
7. Wait for the AMI to become **available** (status: available in **AMIs** section)

✓ This AMI will be used for launching new instances automatically in ASG.

Step 2: Create a Launch Template

1. Go to **EC2** → **instances->Launch Templates** → **Create Launch Template**
2. **Launch Template Name:** my-launch-template
3. **AMI:** Select the AMI created in Step 1 (my-webserver-AMI)
4. **Instance Type:** t2.micro (or your choice)
5. **Key Pair:** Select your existing key pair
6. **Security Groups:** Select the **Load Balancer Security Group** (allow HTTP 80)
7. **User Data:** Optional (already baked into AMI)
8. Click **Create Launch Template**

✓ Launch Template is ready for Auto Scaling.

Step 3: Create Auto Scaling Group (ASG)

1. Go to **EC2** → **Auto Scaling Groups** → **Create Auto Scaling Group**
 2. **Name:** webserver-asg
 3. **Launch Template:** Select my-launch-template
 4. **Template Version:** Select **Version 1**
 5. Click **Next**
-

Step 3.1: Network Settings

1. **VPC:** Default VPC (or your existing VPC)
 2. **Availability Zones (AZs):** Select **all** subnets (for high availability)
 3. **Load Balancer:** Enable **Attach to an existing load balancer**
 - Select your **existing Load Balancer**
 4. **Health Check:** HTTP
 - Set **Health Check Grace Period:** 300 seconds
 5. Click **Next**
-

Step 3.2: Configure Group Size

1. **Desired Capacity:** 2 (initial instances)
 2. **Minimum Capacity:** 1
 3. **Maximum Capacity:** 4
 4. Click **Next**
-

Step 3.3: Review and Create ASG

1. Review all settings
2. Click **Create Auto Scaling Group**

✓ Auto Scaling Group is now ready and attached to your Load Balancer.

Step 4: Verify Load Balancer and Auto Scaling

1. Go to **Load Balancers** → **Select your Load Balancer**
2. Click **Instances** → **Target Group** → **Registered Targets**
 - You will see **2 healthy instances** initially
3. Check **Auto Scaling Group** → **Instances**
 - You may see up to **4 instances** if scaling triggers
4. Test **auto recovery**:
 - **Terminate one of the instances**
 - After a few seconds, **ASG will launch a new instance automatically**
 - Traffic will automatically be **distributed by the Load Balancer**

Step 5: Test the Load Balancer

1. Copy the **Load Balancer DNS Name** from ALB
2. Open in a browser → You should see your web page from one of the instances
3. Refresh multiple times → traffic alternates between instances

✓ This confirms **Auto Scaling + Load Balancer** is working correctly.

WEEK-8---SNS,SQS & S3-SNS-SQS-LAMBDA SERVICES

1) Simple Notification Service (SNS)

A) Create SNS Topic and Send Message to Email

Step 1: Create SNS Topic

1. Login to **AWS Console**
2. In search bar → Type **SNS**
3. Open **Amazon SNS (Simple Notification Service)**
4. In left panel → Click **Topics**
5. Click **Create topic**

Configure:

- **Type** → Select **Standard**
 - **Name** → MyEmailTopic
 - Leave other settings default (unless required)
6. Click **Create topic**
-

Step 2: Create Email Subscription

1. Open your created topic
2. Go to **Subscriptions** tab
3. Click **Create subscription**

Configure:

- **Protocol** → Select **Email**
 - **Endpoint** → Enter your email address
4. Click **Create subscription**
-

Step 3: Confirm Subscription

- Go to your email inbox
- You will receive a mail from **Amazon SNS**
- Click **Confirm subscription**

Now your email is successfully subscribed 📧

Step 4: Publish Test Message

1. Open your SNS topic
2. Click **Publish message**

3. Add:
 - **Subject** → Test Mail
 - **Message body** → Hello this is SNS test
4. Click **Publish message**

You will receive email notification.

B) Trigger Email When S3 Object isUploaded (S3 → SNS → Email)

Now we extend this setup:

Flow:

S3 Bucket → Event Notification → SNS Topic → Email

Step 1: Create S3 Bucket

1. Search → **S3**
2. Click **Create bucket**

Configure:

- **Bucket name** → my-upload-bucket-2026
 - Choose Region (Important: Same region as SNS topic)
 - Keep default settings (unless specific requirement)
3. Click **Create bucket**
-

Step 2: Create SNS Topic (If not already created)

You can reuse the topic from Part A
OR create new topic like:

- Topic name → S3UploadNotification

Make sure email subscription is confirmed.

Step 3: Allow S3 to Publish to SNS (Access Policy)

1. Open SNS Topic
2. Go to **Access policy**
3. Click **Edit**

Add permission for S3 to publish.

If using Basic Editor:

Access Policy that needs to be configured on SNS:

a) replace SNS topic ARN

b) Replace s3 bucket ARN

c) Replace account

```
{  
  "Version": "2012-10-17",  
  "Id": "example-ID",  
  "Statement": [  
    {  
      "Sid": "Example SNS topic policy",  
      "Effect": "Allow",  
      "Principal": {  
        "Service": "s3.amazonaws.com"  
      },  
      "Action": [  
        "SNS:Publish"  
      ],  
      "Resource": "arn:aws:sns:us-east-1:347657230123:snsEventTopic",  
      "Condition": {  
        "ArnLike": {  
          "aws:SourceArn": "arn:aws:s3:::s3bucket-test1007"  
        },  
        "StringEquals": {  
          "aws:SourceAccount": "347657230123"
```

```
}  
  
}  
  
}  
  
]  
  
}
```

(Modern AWS console often auto-generates this when configuring from S3)

Step 4: Configure S3 Event Notification

1. Open your S3 bucket
2. Go to **Properties**
3. Scroll to **Event notifications**
4. Click **Create event notification**

Configure:

- **Event name** → UploadNotification
 - **Event types** → Select:
 - All object create events
 - **Destination:**
 - Select **SNS topic**
 - Choose your topic (S3UploadNotification)
5. Click **Save changes**
-

Step 5: Test It

1. Upload any file into the S3 bucket
2. After upload completes:
 - S3 triggers event
 - SNS receives publish request
 - SNS sends email

You will receive mail about object upload.

Simple Queue Service (SQS)

2) Create SQS Queue and Send Message

We will:

1. Create an SQS Queue
2. Send a message to the Queue
3. Receive the message from the Queue

Step 1: Create SQS Queue

1. Login to **AWS Console**
2. In the **search bar** → **Type SQS**
3. Open **Amazon Simple Queue Service**
4. In the dashboard → Click **Create queue**

Configure:

- **Type** → **Select Standard**
- **Name** → **MyQueue**
- Leave other settings as **default**
- 5. Click **Create queue**

Your **SQS queue** is now created successfully.

Step 2: Send Message to Queue

1. Open the **MyQueue** queue
2. Click **Send and receive messages**
3. In the **Message body**, type:

Hello this is SQS test message

4. Click **Send message**

Your message is now stored in the queue.

Step 3: Receive Message from Queue

1. In the same page (**Send and receive messages**)
2. Click **Poll for messages**

You will see the message displayed.

Example output:

Hello this is SQS test message

3. After processing, click **Delete message** (optional)
-

Result:

- SQS queue created
 - Message sent successfully
 - Message received from queue
-

3) S3 Bucket Event → SNS → SQS → Lambda Processing

S3 bucket event (like file upload) triggers SNS, which sends the notification to SQS. Then **Lambda acts as the consumer and processes the message.**

Services used:

- Amazon S3
 - Amazon Simple Notification Service
 - Amazon Simple Queue Service
 - AWS Lambda
-

Step 1: Create S3 Bucket

1. Login to AWS Console
2. Search **S3**
3. Open **Amazon S3**
4. Click **Create bucket**

Configure:

- Bucket name → **mys3eventbucket123** (must be unique)
- Region → choose your region

Leave other settings default.

5. Click **Create bucket**
-

Step 2: Create SNS Topic

1. Search **SNS** in AWS Console
2. Open **Amazon SNS**
3. Click **Topics** → **Create topic**

Configure:

- Type → **Standard**
- Name → **MyS3SNSTopic**

4. Click **Create topic**

Copy the **SNS Topic ARN**.

Step 3: Create SQS Queue

1. Search **SQS** in AWS Console
2. Open **Amazon SQS**
3. Click **Create queue**

Configure:

- Type → **Standard**
- Name → **MyS3Queue**

4. Click **Create queue**

Copy the **Queue ARN**.

Step 4: Subscribe SQS to SNS

1. Open **MyS3NSTopic**
2. Click **Create subscription**

Configure:

- Protocol → **Amazon SQS**
- Endpoint → Select **MyS3Queue**

3. Click **Create subscription**

Step 5: Add Access Policy to SQS

Open **MyS3Queue** → **Access Policy** → **Edit**

Replace:

- SQS ARN
- SNS ARN

```
{  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Principal": {  
        "Service": "sns.amazonaws.com"  
      },  
      "Action": "sqs:SendMessage",  
      "Resource": "arn:aws:sqs:us-east-1:347657230123:s3TestQueue",  
      "Condition": {  
        "ArnEquals": {  
          "aws:SourceArn": "arn:aws:sns:us-east-1:347657230123:snsEventTopic"  
        }  
      }  
    }  
  ]  
}
```



```
]
}
```

Click **Save**.

Step 6: Add Access Policy to SNS

Open **SNS Topic** → **Edit access policy**

Replace:

- SNS topic ARN
- S3 bucket ARN
- Account ID

```
{
  "Version": "2012-10-17",
  "Id": "example-ID",
  "Statement": [
    {
      "Sid": "Example SNS topic policy",
      "Effect": "Allow",
      "Principal": {
        "Service": "s3.amazonaws.com"
      },
      "Action": [
        "SNS:Publish"
      ],
      "Resource": "SNS topic ARN",
      "Condition": {
        "ArnLike": {
```

```
    "aws:SourceArn": "S3 bucket ARN"

  },

  "StringEquals": {

    "aws:SourceAccount": "Account ID"

  }

}

}

]

}
```

Save the policy.

Step 7: Configure S3 Event Notification

1. Open **S3 Bucket (mys3eventbucket123)**
2. Go to **Properties**
3. Scroll to **Event notifications**

Click **Create event notification**

Configure:

- Event name → **S3UploadEvent**
- Event type → **All object create events**

Destination:

- **SNS Topic** → **MyS3SNSTopic**

Click **Save changes**.

Step 8: Test the Architecture

1. Open the **S3 bucket**
2. Click **Upload file**
3. Upload any file (example: **test.txt**)

Step 9: Verify Message in SQS

1. Open **MyS3Queue**
2. Click **Send and receive messages**
3. Click **Poll for messages**

You will see the **S3 event notification message**.

Step 10: Configure Lambda as Consumer

Now configure Lambda to **automatically process messages from SQS**.

Create Lambda Function

1. Search **Lambda** in AWS Console
2. Open **AWS Lambda**
3. Click **Create Function**

Configure:

- Author from scratch
- Function name → **SQSConsumerFunction**
- Runtime → **Python 3.x**

Click **Create function**

Add SQS Trigger

1. Open **SQSConsumerFunction**
2. Click **Add Trigger**
3. Select **SQS**
4. Choose **MyS3Queue**

Click **Add**

Now Lambda will automatically read messages from SQS.

Lambda Code

Replace code with:

```
def lambda_handler(event, context):
```

```
    for record in event['Records']:
```

```
        print("Message received from SQS:")
```

```
        print(record['body'])
```

```
return {  
  
    'statusCode': 200,  
  
    'body': 'Message processed successfully'  
  
}
```

Click **Deploy**.

Result

When a file is uploaded to S3:

1. S3 sends a notification to SNS
2. SNS forwards the message to SQS
3. SQS stores the message
4. Lambda automatically reads and processes the message

This creates a **serverless event-driven architecture commonly used in real cloud applications**.

WEEK-8: S3 → SNS → SQS → Lambda Architecture

Step 1: Create S3 Bucket

1. Go to **Amazon S3**
 2. Click **Create Bucket**
 3. Configure:
 - Bucket name → harshitha-bucket-sns
 - Choose Region
 - Keep default settings
 4. Click **Create Bucket**
-

Step 2: Create SNS Topic

1. Open **Amazon SNS**
 2. Click **Topics → Create Topic**
 3. Configure:
 - Type → Standard
 - Name → HarshithaTopic
 4. Click **Create Topic**
-

Step 3: Create SQS Queue

1. Open **Amazon SQS**
 2. Click **Create Queue**
 3. Configure:
 - Type → Standard
 - Name → harshithaQueue
 4. Click **Create Queue**
-

Step 4: Subscribe SQS to SNS

1. Open **HarshithaTopic**
 2. Click **Create Subscription**
 3. Configure:
 - Protocol → Amazon SQS
 - Endpoint → harshithaQueue
 4. Click **Create Subscription**
-

Step 5: Add Access Policy to SQS

1. Open SQS Queue → **Access Policy → Edit**

2. Replace:

- SQS ARN
- SNS ARN

```
{
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "sns.amazonaws.com"
      },
      "Action": "sqs:SendMessage",
      "Resource": "YOUR_SQS_ARN",
      "Condition": {
        "ArnEquals": {
          "aws:SourceArn": "YOUR_SNS_ARN"
        }
      }
    }
  ]
}
```

3. Click **Save**

Step 6: Add Access Policy to SNS

1. Open SNS Topic → **Edit Access Policy**

2. Replace:

- SNS Topic ARN
- S3 Bucket ARN
- Account ID

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "s3.amazonaws.com"
      },
      "Action": "SNS:Publish",
      "Resource": "YOUR_SNS_ARN",
      "Condition": {
        "ArnLike": {
          "aws:SourceArn": "YOUR_S3_BUCKET_ARN"
        }
      },
      "StringEquals": {
        "aws:SourceAccount": "YOUR_ACCOUNT_ID"
      }
    }
  ]
}
```

```
}  
}  
}  
]  
}
```

3. Click **Save**

Step 7: Configure S3 Event Notification

1. Open S3 Bucket
2. Go to **Properties** → **Event Notifications**
3. Click **Create Event Notification**
4. Configure:
 - Event name → S3UploadEvent
 - Event type → All object create events
 - Destination → SNS Topic (HarshithaTopic)
5. Click **Save**

Step 8: Test the Flow

1. Upload a file (e.g., test.txt) to S3

Flow happens automatically:

S3 Upload → SNS → SQS

Step 9: Verify Message in SQS

1. Open SQS Queue
2. Click **Send and Receive Messages**
3. Click **Poll for Messages**

✓ You will see the S3 event message

Step 10: Create Lambda Function

1. Open **AWS Lambda**
2. Click **Create Function**
3. Configure:
 - Author from scratch
 - Name → HarshithaSQSConsumerFunction
 - Runtime → Python

4. Click **Create**
-

Step 11: Add SQS Trigger to Lambda

1. Open Lambda function
 2. Click **Add Trigger**
 3. Select **SQS**
 4. Choose harshithaQueue
 5. Click **Add**
-

Step 12: Add Lambda Code

Replace with:

```
def lambda_handler(event, context):
    for record in event['Records']:
        print("Message received from SQS:")
        print(record['body'])

    return {
        'statusCode': 200,
        'body': 'Message processed successfully'
    }
```

Click **Deploy**

RESULT:

When file is uploaded:

1. S3 sends notification to SNS
 2. SNS forwards to SQS
 3. SQS stores message
 4. Lambda processes message automatically
-

Optional: Email Notification via SNS

Step 1: Create Email Subscription

1. Open SNS Topic
 2. Go to **Subscriptions** → **Create Subscription**
 3. Configure:
 - Protocol → Email
 - Endpoint → Your email
-

Step 2: Confirm Subscription

- Check inbox
 - Click **Confirm Subscription**
-

Step 3: Send Test Message

1. Open SNS Topic
2. Click **Publish Message**
3. Add:
 - Subject → Test Mail
 - Message → Hello this is SNS test
4. Click **Publish**

✓ You will receive an email